Transformer 前置知识

> 一份零基础友好的 Transformer 入门指南。读完你就能看懂 QKV、多头注意力、位置编码、FFN、

> 训练推理差异、分词机制，然后再去读横纵分析报告就会顺畅很多。

>

> **撰写时间**：2026-04-29

目录

1. 为什么要懂 Transformer?
2. 前 Transformer 时代：RNN 帝国为什么倒了
3. 注意力机制：Query、Key、Value 到底是什么
4. 多头注意力：为什么需要八个头
5. 位置编码：为无序的计算注入顺序
6. FFN 层：被低估的记忆仓库
7. 为什么 Transformer 能 scale
8. 训练与推理：Transformer 的双面人生
9. 分词：Transformer 怎么「识字」
10. 一张图谱串起来
11. 与 MoE 前置知识的对照
12. 延伸阅读

一、为什么要懂 Transformer?

2026 年，你用的每一个 AI 产品背后几乎都站着 Transformer——GPT-4、Claude、Gemini、DeepSeek、文心一言、通义千问。它不只是大语言模型的基础，还是视觉（ViT）、语音（Whisper）、蛋白质折叠（AlphaFold）等 AI 前沿的通用架构。

这篇文章的目标读者是**没有深度学习背景的人**。全部用大白话讲，不讲数学推导，只说直觉和关键概念。读完之后你会理解：

- Transformer 跟以前的 RNN/LSTM 到底区别在哪
- Q、K、V 是什么，为什么 softmax 要除 √d_k
- 为什么需要那么多"头"
- 训练和推理的本质差异，KV Cache 是干什么的
- 分词（Tokenizer）这个小细节为什么能决定模型天花板

文中会不断跟 **MoE（混合专家模型）** 做对比参照——因为两者共享"软路由 + 记忆检索"的核心隐喻，对照着看会豁然开朗。

二、前 Transformer 时代：RNN 帝国为什么倒了

2017 年之前，序列建模是 RNN 的天下。一条句子进来，RNN 从左到右逐个 token 吞下，每一步输出一个隐状态 h_t，传给下一步。LSTM 和 GRU 给 RNN 加了门控机制，缓解了梯度消失——你能记住 20 个 token 之前的"主语是单数"这种信息了。

但有个绕不过去的硬伤：**串行计算**。

要算 h_{100}，必须先把前面的 99 步都算完。GPU 几千个核心，用不上。训练一条长句子，要等 CPU 时代的逻辑在 GPU 上慢慢挪。更致命的是，虽然 LSTM 能记住 20 步前，到 100 步、200 步，该忘还是忘——反向传播路径太长，梯度信号被洗成噪声。

Seq2Seq 架构（Sutskever et al., 2014）给这个问题打了个补丁：用一个定长向量 c 代表整个源句，decoder 从 c 解码。问题显而易见——"定长向量压缩整句"本身就是信息瓶颈。句子一长，前半句的信息被挤掉了。

注意力机制（Bahdanau et al., 2014）是第一个真正的突破：不再依赖一个定长向量，decoder 每一步都**动态搜索**整个源句，自己去挑"现在该看哪些词"。这是关键的思想跃迁——从"被动接收一个压缩表示"变成了"主动检索"。这个"软寻址"的视角，成了理解 Transformer 的核心隐喻。

> **MoE 对照**：MoE 预训练的历史同样经历了"从全量计算到条件计算"的跃迁——从所有样本经过所有参数（Dense），到每个 token 只激活部分专家。两者底层逻辑完全一致：**用软路由取代全量搬运**。

三、注意力机制：Query、Key、Value 到底是什么

自注意力（Self-Attention）是 Transformer 的心脏。

一句话说明白：**让每个 token 去问其他 token："你跟我什么关系？"然后根据关系强弱，加权汇总其他 token 的信息。**

三个矩阵 W_Q、W_K、W_V 做的是同一件事的不同阶段：

- **Query（查询）**：当前 token 在问什么。比如"我是动词 'eat'，我需要知道谁在吃、吃了什么"
- **Key（键）**：每个 token 提供什么信息标签。比如"我是名词，我可以当主语"
- **Value（值）**：每个 token 携带的实质信息。比如"我的具体语义是 'cat'"

计算过程：

```
Attention(Q, K, V) = softmax(QK^T / √d_k) × V
```

QK^T 算出注意力分数矩阵——每个位置对其他所有位置的"关注度"。除以 √d_k 是因为维度一大，点积值爆炸，softmax 会进入梯度几乎为零的区域。softmax 归一化成概率分布。乘以 V，加权汇总。这一层的输出是"所有 token 的信息按关注度重新调配的结果"。

这个视角跟检索系统一模一样：Query 是搜索词，Key 是文档索引，Value 是文档内容。跟 MoE 的 router 也很像——MoE 用门控网络决定"每个 token 去哪几个 expert"，Transformer 用 QK 相似度决定"每个 token 关注哪几个 token"。都是**软路由**，只是路由的对象不同：Transformer 路由信息，MoE 路由计算。

一张图理解 QKV（小白视角）

想象你要翻译："The cat sat on the mat because it was tired."

当模型处理 "it" 这个 token 时：

```
"it" 的 Query = "我正在找一个单数名词，我可能是代词指代"
所有 token 的 Keys = ["The": 冠词, "cat": 单数名词, "sat": 动词, ...]
"cat" 的 Key 跟 "it" 的 Query 匹配度最高 → Attention 分数最大
"cat" 的 Value = "猫" → 被大部分权重传递给 "it"
```

结果：模型"知道"了 it 指的是 cat，而不是 mat。这个能力叫**指代消解**（Coreference Resolution），在传统 NLP 里是一个单独的任务，Transformer 把注意力做到位了就自然而然地解决了。

Cross-Attention vs Self-Attention

上面说的是 Self-Attention（序列自己看自己）。Encoder-Decoder 架构里还有一种 Cross-Attention：

- **Self-Attention**：序列内部的 token 互看。编码器能看整句、解码器只能看已生成部分
- **Cross-Attention**：Decoder 的 Query 去检索 Encoder 的所有 Key/Value，问"源句里哪个词跟我要生成的下一个词最相关？"
- 这跟 MoE 的门控网络逻辑一致——都是「根据当前需要，从候选池里挑最相关的信息」

Decoder-only 架构（GPT 系）之所以成为主流，正是因为**把 Cross-Attention 省掉了**——所有知识都包装在 Self-Attention + FFN 里，架构更统一，推理也更简洁。

四、多头注意力：为什么需要八个头

单头注意力的表达能力有限——一个 token 在一次前向中只能形成一种"关系模式"。但实际上，一个 token 可能需要同时关注"语法结构"（动词接什么介词）和"语义角色"（谁是施事者）。

多头注意力的做法：把 Q、K、V 分别投影到多个低维子空间（比如 8 个头、每个头 d_k = 512/8 = 64），每个头独立计算注意力，最后拼接。

```
MultiHead(Q, K, V) = Concat(head_1, ..., head_h) × W_O
```

效果是，不同头确实学会了关注不同的关系模式。有的头负责相邻词之间的句法依赖，有的头负责长距离语义关联，有的头专门看标点符号。这不是设计出来的——是训练出来的涌现行为。

> **MoE 对照**：多头注意力很像"在注意力维度上做了细粒度专家"——每个头是一个专门的关系检测器，它们并行工作、最后融合。这也解释了为什么后来 DeepSeek 的 MLA 能用低秩压缩减少 KV cache：多个头之间的信息有大量重叠，可以被压缩进一个更小的潜空间。这和 MoE 把一个大 FFN 拆成多个小 expert 的"细粒度"思路异曲同工。

五、位置编码：为无序的计算注入顺序

RNN 天然知道顺序——按序输入。Transformer 并行喂入所有 token，序列顺序完全丢失了。"我爱北京天安门"和"门安天京北爱我"对这层矩阵来说没区别。

位置编码就是给每个 token 打上"我是第 i 个"的标签。四代方案演进：

方案	做法	优点	缺点
Sinusoidal（原始论文）	sin/cos 函数编码	理论优雅，可线性外推	实际效果一般
Learned（BERT 用）	可训练的 embedding 查表	简单有效	无法外推到训练长度之外
RoPE（Su et al., 2021）	对 Q、K 向量做旋转	天然编码相对距离，外推性好	长序列外推仍需调整
ALiBi（Press et al., 2021）	直接在注意力分数上加偏置	极简，外推性最好	精度略低于 RoPE

RoPE 为什么赢？ 它不对输入向量加位置编码，而是对注意力计算的 Q 和 K 向量做**旋转**——旋转角度由位置决定。这样 Q^T K 的点积值天然包含了相对位置信息（因为旋转矩阵的性质：Rot(m)^T Rot(n) = Rot(n−m)）。LLaMA、Mistral、Qwen、DeepSeek 全都采用了 RoPE。

六、FFN 层：被低估的记忆仓库

每层 Transformer 有两个子层：Multi-Head Attention 和 Feed-Forward Network。大家在聊 Attention，很少聊 FFN。

但 **FFN 占了 Transformer 总参数的三分之二**。Geva et al.（2021）给出过一个漂亮的观点——FFN 是一个 key-value 记忆库：

```
第一层线性变换 W₁：检测"现在输入里有什么模式？"
激活函数：阈值，"这个模式激活了吗？"
第二层线性变换 W₂：输出"这个模式对应的知识"
```

所以 Attention 负责"从上下文检索相关信息"，FFN 负责"从参数记忆中提取知识"。Attention 回答"这段文本里说了什么？"，FFN 回答"根据你的训练知识，接下来该是什么？"。

> **MoE 的切入点恰恰在这里**。既然 FFN 是一个记忆库，为什么每个 token 都要查询整个记忆库？能不能让 token 只查询最相关的几个"记忆分区"？这就是 MoE-Transformer 的核心动机：把 FFN 拆成多个 expert，每个 token 通过门控网络只激活 1-2 个 expert。计算量不变（激活参数不变），但总参数量可以大幅增加——模型"知道"的更多，"花的钱"一样。

七、为什么 Transformer 能 scale

Scaling 的前提是三个能力，少一个都不行：

1. **完全并行**：训练时每个 token 同时看到所有上下文，GPU 全部核心都在干活。对比 RNN 的串行瓶颈——1024 个 token 要串行 1024 步——这是代际差异。
1. **恒定路径长度**：任意两个 token 之间的信息路径都是 O(1)——一层 attention 就能直接传递。RNN 要走过整个序列长度，梯度衰减严重。这是长距离依赖能力的根因。
1. **结构极度规整**：所有层结构相同、所有位置处理方式相同。想加参数？加层、加宽、加头——没有架构瓶颈挡路。RNN 同样可以堆叠，但梯度消失限制了深度；Transformer 的残差连接 + LayerNorm 让几百层的网络也能稳定训练。

这三个能力叠加，再加上 Scaling Laws 的发现——模型性能和算力、数据、参数量呈平滑幂律关系——让 Transformer 成了第一个真正"投入更多算力就能稳定变强"的架构。RNN 加算力效果不明确，Transformer 加算力稳定提升。这是它九年统治力的根源。

八、训练与推理：Transformer 的双面人生

很多人学了整体架构，但搞不清训练和推理时发生了什么**本质不同**。这是面试最常踩的坑，也是理解"为什么推理优化这么重要"的关键。

训练时（Teacher Forcing）

所有 token 同时进入模型，一次前向计算出所有位置的预测。因为 Ground Truth 已知（"the cat sat"），模型可以并行计算：

```
输入:  "[BOS] the cat sat"
目标:  "the cat sat [EOS]"
一次前向，对所有 token 位置求交叉熵 loss
用 Masked Self-Attention 防止偷看未来 token（attention 矩阵上三角置 -∞）
```

推理时（自回归生成）

模型只有一个初始 token "[BOS]"，然后进入循环：

```
t=1: 输入 [BOS]           → 输出概率分布 → 采样 "the"
t=2: 输入 [BOS, the]      → 输出概率分布 → 采样 "cat"
t=3: 输入 [BOS, the, cat] → 输出概率分布 → 采样 "sat"
t=4: 输入 [BOS, the, ...] → 输出概率分布 → 采样 [EOS] → 停止
```

每一步都要重新跑完整前向——这是为什么长序列推理慢。

KV Cache：推理加速的关键

推理时，每次新 token 进来都要重新计算之前所有 token 的 K 和 V。这纯属浪费——"the"的 Key 和 Value 在上一步已经算过了。

KV Cache 的做法：**把每一步算出的 K、V 保存在显存里**，下一步直接用缓存，只算新 token 的。

代价：KV cache 本身也要吃显存。长序列的 KV cache 可能比模型参数还大——LLaMA 70B 在 8K token 的 KV cache 要占 ~20GB，加上模型本身的 140GB，一张 H100 80GB 就爆了。

所以 GQA、MQA、MLA 这些注意力压缩方案如此重要——核心目标是**缩减 KV cache 大小**，而不是缩减计算量。做推理优化的工程师心里，瓶颈顺序是：显存带宽 > 计算吞吐 > 模型大小。

九、分词：Transformer 怎么「识字」

Transformer 处理的不是单词、不是汉字，是 **token**。Tokenizer 把原始文本切成 token 序列。

BPE（Byte Pair Encoding） 是主流方案：从单个字符开始，统计出现频率最高的相邻字符对，合并成一个新 token。反复执行直到词表满。结果是高频词自成一个 token（"the" → 1 个 token），低频词拆成子词（"transformer" → "trans" + "form" + "er"）。

中文分词一般用字符级 BPE 或 SentencePiece。一个汉字通常是 1–2 个 token，一个英文普通单词也是 1–2 个 token。

分词为什么重要？ 三个方面：

- **感知粒度**："pheromone"（信息素）如果被切成 phe + rom + one，模型就无法感知这个概念——它看到的是三个无意义的子词
- **模型成本**：词表大小直接影响 embedding 层参数量。词表 32K vs 128K，embedding 层参数差 4 倍
- **跨模型不通用**：LLaMA 的 tokenizer 切出来的 token 和 GPT-4 的不一样，不能混用。做模型对比时这是最容易忽略的维度

十、一张图谱串起来

```
文本输入
  ↓ Tokenizer → 分词成 token 序列
  ↓ Embedding + Positional Encoding → 向量表示
  ↓
  ↓ [开始 1 层 Transformer]
  ↓   ├─ Multi-Head Self-Attention（QKV → 加权信息聚合）
  ↓   │     └─ 跟 MoE Router 同源：「软路由」哲学
  ↓   ├─ Add & LayerNorm（残差连接 + 归一化）
  ↓   ├─ FFN（Key-Value 记忆库 → 提取知识）
  ↓   │     └─ MoE 替代点：把一个大 FFN 拆成多个 expert
  ↓   └─ Add & LayerNorm
  ↓ [重复 N 层]
  ↓
  ↓ 输出层 → 映射到词表 → softmax → 采样下一个 token
```

十一、与 MoE 前置知识的对照

这份文档和 MoE 前置知识文档是**镜像设计**。对照阅读可以加深两条线的理解：

概念	Transformer 视角	MoE 视角
软路由	Attention 用 QK 相似度路由信息	Gate/Router 用门控网络路由计算
并行处理	多头注意力并行检测多种关系	多个 Expert 并行处理不同知识域
专业化	不同 attention head 学习不同依赖模式	不同 expert 专注不同领域知识
压缩冗余	MLA 低秩压缩 KV cache	细粒度专家减少参数冗余
条件计算	Cross-Attention 按需检索源句信息	Top-k 路由只激活最相关 expert
记忆架构	FFN = Key-Value 记忆库	每个 Expert = 独立的知识分区

两者共同的核心命题：**让每个 token 更精准地获得它"需要的信息"（Transformer）和"需要的计算"（MoE）。**

十二、延伸阅读

- Transformer 横纵分析报告 — 纵向历史 + 横向竞争 + 面试/场景/应用题
- MoE 前置知识 — MoE 混合专家模型的入门指南（如有）

核心论文

- Vaswani et al., "Attention Is All You Need" (2017) — https://arxiv.org/abs/1706.03762
- Bahdanau et al., "Neural Machine Translation by Jointly Learning to Align and Translate" (2014) — https://arxiv.org/abs/1409.0473
- Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers" (2018) — https://arxiv.org/abs/1810.04805
- Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding" (2021) — https://arxiv.org/abs/2104.09864
- Geva et al., "Transformer Feed-Forward Layers Are Key-Value Memories" (2021) — https://arxiv.org/abs/2012.14913
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention" (2022) — https://arxiv.org/abs/2205.14135

下一篇：Transformer 横纵分析报告 — 从这里直接进入完整的纵向历史分析和横向竞争对比。